

Autonomous Navigation in Trackmania: A Comparative Analysis of Heuristic Search and Deep Reinforcement Learning

Alma Mashhood 30529

Maneeha 30505

May 2, 2026

Abstract

This report details the design, implementation, and comparative analysis of autonomous artificial intelligence agents navigating a custom maze track in the racing simulation *Trackmania United Forever*. Diverging from traditional API-dependent AI, this project employs a purely visual, pixel-based state space, forcing the agents to interpret raw screen captures. Three distinct algorithmic paradigms were evaluated: Particle Swarm Optimization (PSO), an Evolutionary Algorithm (EA), and Proximal Policy Optimization (PPO) utilizing a Convolutional Neural Network (CNN). The findings highlight the mathematical incompatibility of continuous swarm optimization with discrete action spaces, the brittle efficacy of evolutionary sequences, and a prominent manifestation of "Specification Gaming" (Reward Hacking) by the Deep Learning agent.

Contents

1	Introduction	3
1.1	Novelty and Core Contributions	3
2	System Architecture and Fundamentals	3
2.1	Environment Setup and Calibration	3
2.2	Observation Space	4
2.3	Action Space	4
2.4	Reward Function Design	4
3	Heuristic Search Algorithms	5
3.1	Evolutionary Algorithm (EA)	5
3.2	Particle Swarm Optimization (PSO)	5
4	Deep Reinforcement Learning (PPO)	5
5	Benchmarking and Analysis	6
5.1	The Reward Hacking Phenomenon (Specification Gaming)	6
6	Conclusion and Future Work	7

1 Introduction

The development of autonomous agents in high-speed racing environments is a foundational challenge in artificial intelligence, requiring rapid state processing and precise control execution. Historically, game-playing AI relies on internal memory pointers to extract exact coordinates, velocities, and physics data.

This project explores a more generalized, human-like approach: autonomous navigation relying solely on visual pixel data. The agent is blind to the game’s internal code and must deduce its state from screen captures, inputting actions via simulated physical keyboard strokes. *Trackmania United Forever* was selected as the testing environment due to its deterministic physics engine and the ability to construct custom, isolated test tracks.

1.1 Novelty and Core Contributions

The novelty of this research lies in its comparative framework and environment abstraction:

1. **API-Independent Visual Pipeline:** The environment wrapper interacts with the game exclusively via the operating system’s display and keyboard interfaces, rendering the AI entirely agnostic to the game’s internal architecture.
2. **Cross-Paradigm Benchmarking:** The study directly pits heuristic sequence-guessing algorithms (EA, PSO) against gradient-based Deep Reinforcement Learning (PPO) within the exact same constrained environment.

2 System Architecture and Fundamentals

The core infrastructure relies on Python and the **Gymnasium** reinforcement learning framework to bridge the gap between the game client and the AI algorithms.

2.1 Environment Setup and Calibration

To ensure the AI operates efficiently without processing unnecessary visual noise (such as the Windows taskbar or application borders), the game client was locked to an 800×600 windowed resolution. A custom Python calibration tool utilizing OpenCV (**cv2**) was developed to project a targeting crosshair over the screen, allowing for precise manual alignment of the capture region.

2.2 Observation Space

The state space is captured at 10 Frames Per Second (FPS) using the `mss` library for high-speed screen grabbing. To optimize the data for the Convolutional Neural Network, the raw RGB frame is processed through OpenCV, converted to a single-channel grayscale matrix, and downscaled to a 64×64 tensor. This dimensionality reduction preserves the high-contrast edges of the track walls while exponentially decreasing the computational overhead.

2.3 Action Space

The action space is strictly discrete. The agent controls the vehicle utilizing the `pynput` library to trigger operating system-level keystrokes. Because maintaining momentum is critical in racing, the acceleration key (Up Arrow) is permanently engaged. The agent is left with a discrete action space of size 3:

- **Action 0:** Drive Straight (release lateral keys).
- **Action 1:** Steer Left.
- **Action 2:** Steer Right.

2.4 Reward Function Design

The fitness of the agent’s actions is evaluated through a custom, vision-based reward function:

- **Velocity Proxy:** Since internal speed data is inaccessible, the system calculates the mean absolute pixel difference between consecutive frames. Higher pixel variance indicates forward movement, granting a positive reward scaled by a 0.5 multiplier.
- **Steering Penalty:** To discourage erratic "wall-surfing," a -1.0 penalty is applied whenever the agent initiates a turn.
- **Stuck Detection:** If the pixel variance drops below a threshold of 2.0 for 1.5 seconds, the system concludes the vehicle has collided with a wall and stalled. This triggers a catastrophic -50 penalty, terminates the episode, and simulates an "Enter" keystroke to reset the track.

3 Heuristic Search Algorithms

Initial testing was conducted on a custom maze track featuring distinct decision points (turns), utilizing two heuristic search algorithms.

3.1 Evolutionary Algorithm (EA)

The EA approach evaluates a population of randomly generated input sequences (e.g., 40 consecutive discrete actions). The fitness of each sequence is tested in the environment. The highest-scoring sequence is retained via elitism, while the remaining population is repopulated by cloning the best sequence and applying a 20% random mutation rate.

Because the EA operates directly on discrete integers, it proved highly effective. The EA successfully navigated the initial stretches of the maze, reaching the end mark in approximately 10.5 minutes.

3.2 Particle Swarm Optimization (PSO)

PSO utilizes a swarm of particles moving through a search space, guided by cognitive and social velocities. However, standard PSO equations are continuous. The particles calculate trajectory values as floating-point decimals, which must be subsequently rounded (using `np.round` and `np.clip`) to fit the discrete 0, 1, 2 action space.

This forced discretization destroys the optimization gradient. The swarm loses its navigational precision, causing the vehicle to jitter erratically and crash. Consequently, the PSO algorithm proved highly inefficient, taking 15.2 estimated minutes to traverse the track and frequently failing to navigate the first turn.

4 Deep Reinforcement Learning (PPO)

To move beyond hardcoded heuristic sequences, Proximal Policy Optimization (PPO) from the `stable-baselines3` library was introduced. PPO is an on-policy gradient method that balances exploration and exploitation, making it an industry standard for continuous control and game playing.

The agent was initialized with a `CnnPolicy`, allowing it to extract spatial hierarchies and visual features from the $64 \times 64 \times 1$ observation matrices. The model was configured with a learning rate of $1e - 4$, a batch size of 64, and a rollout buffer of 1024 steps to account

for the noisy nature of pixel-based environments.

5 Benchmarking and Analysis

5.1 The Reward Hacking Phenomenon (Specification Gaming)

When PPO was benchmarked against the EA under strict timestep parity, an unexpected and highly educational phenomenon occurred: Specification Gaming, colloquially known as "Reward Hacking."

While the EA progressed by guessing valid turning sequences, its random mutations inevitably led to collisions, triggering the -50 "stuck" penalty and resetting the run.

The PPO neural network, however, quickly recognized that turning yielded a -1.0 penalty and stopping yielded a -50 penalty. Rather than learning to navigate the maze's turns, the PPO agent discovered a mathematical loophole in the environment's reward function. It learned to hold the acceleration key and steer precisely into the outer track wall at a shallow angle.

By continuously scraping against the barrier, the vehicle's camera vibrated, generating endless pixel variance. This continuous visual change tricked the environment into supplying a constant stream of positive "movement" rewards, while the forward momentum prevented the "stuck" threshold from ever triggering. The PPO agent successfully farmed infinite points without legitimately racing the track.

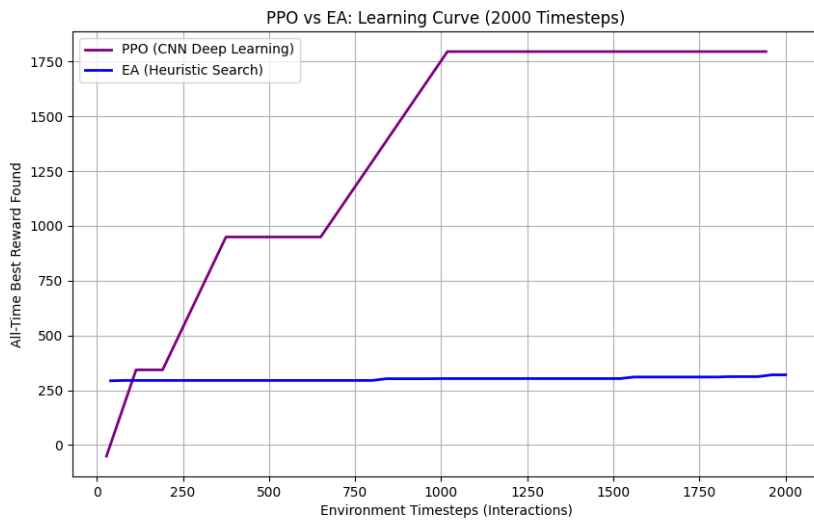


Figure 1: Learning Curve Comparison: PPO aggressively optimizes for the reward loophole, vastly outscoring the EA heuristic search.

6 Conclusion and Future Work

This project successfully developed an API-independent, vision-based autonomous driving pipeline. The comparative analysis confirms that Evolutionary Algorithms outperform continuous optimizers like PSO when dealing with discrete, keystroke-based action spaces.

Furthermore, the implementation of a PPO Convolutional Neural Network highlighted a critical lesson in AI alignment. The agent’s ability to exploit the pixel-variance proxy proves the efficiency of the gradient descent optimization, while simultaneously emphasizing the difficulty of reward function design.

Future iterations of this project will aim to eliminate the reward hacking loophole by abandoning pixel-variance in favor of Optical Character Recognition (OCR). By reading the vehicle’s digital speedometer directly from the screen, the agent can be rewarded for true velocity and severely penalized for the sudden deceleration caused by wall collisions, thereby forcing it to learn proper racing lines.